# Compile-Planned Distributed Training (CPDT)

## Compiler-Optimized ZeRO Sharding, Precision-Adaptive Optimizers, and Static Expert Routing for NeuralScript

**Authors:** Brandon Wiemer
**Date:** April 2026
**Status:** Research Proposal

---

### Abstract

We propose **Compile-Planned Distributed Training (CPDT)**, a unified framework where the NSL compiler statically plans three interconnected distributed training concerns — ZeRO-style state sharding, per-parameter optimizer precision, and MoE expert routing — as a single joint optimization problem solved at compile time. In existing frameworks, each concern is configured independently by the user (DeepSpeed JSON configs, bitsandbytes per-parameter overrides, MoE capacity factors), with no tool capable of jointly optimizing all three. CPDT uses NSL's cost model, memory planner, and weight analysis to compute a **training plan** — a complete specification of what gets sharded where, which optimizer states use 8-bit vs 32-bit precision, and how experts are allocated across devices — before any training begins. The compiled binary contains this plan baked into its communication schedule, kernel configurations, and memory layout, with zero runtime decision overhead.

---

# Part I: Compile-Time ZeRO Strategy Selection

## 1. The Configuration Problem

ZeRO has three stages, each with different memory/communication tradeoffs. The optimal stage depends on model size, GPU count, interconnect bandwidth, batch size, sequence length, and activation memory. Currently, users determine this through:

1. Trial and error (run training, OOM → increase ZeRO stage)
2. Back-of-envelope calculations (parameter count × 16 bytes / GPU count)
3. Framework-specific heuristics (DeepSpeed's auto-tuning, which still requires runtime profiling)

Recent work (AMSP/Lins, AsyncHZP) shows that **per-component independent sharding** — where parameters, gradients, and optimizer states each choose their own sharding group — outperforms uniform ZeRO stages by up to 56% MFU improvement. But this explodes the configuration space: 3 components × multiple sharding group sizes = thousands of possible configurations.

## 2. The Compilation Oracle for ZeRO

NSL's compiler can analytically evaluate any ZeRO configuration:

```
Input:
  - Model AST (parameter sizes, layer topology, compute graph)
  - Cluster spec: N GPUs, intra-node bandwidth, inter-node bandwidth
  - Training config: batch size, sequence length, accumulation steps

For each candidate ZeRO config (s_p, s_g, s_os, mesh_p, mesh_g, mesh_os):
  1. Memory per GPU:
     params_memory = Σ |W_l| × dtype_bytes / s_p        (per shard)
     grad_memory   = Σ |W_l| × dtype_bytes / s_g        (per shard)
     optim_memory  = Σ |W_l| × optim_multiplier / s_os   (per shard)
     activation_memory = memory_planner.peak(model, batch/N, seq)
     total = params + grad + optim + activation

  2. Communication volume per step:
     forward_allgather  = Σ |W_l| × dtype_bytes × (s_p > 1)
     backward_reducescatter = Σ |W_l| × dtype_bytes × (s_g > 1)
     optim_allgather    = Σ |W_l| × dtype_bytes × (s_os > 1)

  3. Communication time (overlapped):
     for each layer l in reverse topo order:
       compute_time_l = cost_model.backward_time(l, batch/N, seq)
       comm_time_l = reducescatter_time(|W_l|, mesh_g, bandwidth)
       overlap_efficiency = min(1.0, compute_time_l / comm_time_l)
       exposed_comm_l = comm_time_l × (1 - overlap_efficiency)

  4. Total step time = forward_compute + backward_compute + exposed_comm +
optim_time

Output: (step_time, memory_per_gpu, communication_volume)
```

**Search space pruning:** The compiler first eliminates infeasible configurations (memory exceeds GPU capacity), then ranks feasible ones by step time. The search is exhaustive for small cluster configs (⩽64 combinations for typical 2-level hierarchical meshes) or uses the cost model's gradient to do directed search for larger clusters.

**Time to solve:** ~200ms for a full search across all feasible configurations. Compare to AMSP's runtime profiling approach (minutes) or DeepSpeed's auto-tuning (hours).

### 2.1 Layer-Granular Communication Scheduling

The compiler doesn't just choose a ZeRO stage — it generates a **per-layer communication schedule** that maximizes overlap:

```
Compiled backward pass (ZeRO-3 with optimal scheduling):

Layer 7 backward:
  LAUNCH async: allgather(W_6_shard)     ← prefetch layer 6 params
  COMPUTE: grad_7 = backward(layer_7, activations_7)
  LAUNCH async: reducescatter(grad_7)    ← scatter while next layer computes

Layer 6 backward:
  WAIT: allgather(W_6_shard)             ← should already be done
  LAUNCH async: allgather(W_5_shard)     ← prefetch layer 5
  COMPUTE: grad_6 = backward(layer_6, activations_6)
  FREE: W_6_full                         ← discard gathered params immediately
  LAUNCH async: reducescatter(grad_6)

...
```

**NSL advantage:** This schedule is embedded in the compiled backward function, not implemented via runtime hooks (`register_post_accumulate_grad_hook`). The communication calls are interleaved with compute calls at the IR level, enabling the Cranelift backend to schedule them optimally. There is no Python-level dispatch overhead between layers.

#### 2.2 Topology-Aware Mesh Selection

When pre-trained weights are available, the compiler analyzes per-layer parameter sizes and computation costs to determine optimal mesh assignments:

- **Large, compute-heavy layers** (FFN projections): Shard parameters across all GPUs (maximize memory savings, high compute hides communication)
- **Small, latency-sensitive layers** (norms, embeddings): Replicate within node (intra-node bandwidth is 10-20× inter-node)
- **Attention layers**: Parameters sharded intra-node only (GQA reduces KV head sharding needs)

This per-layer mesh assignment is a compile-time constant — no runtime decision overhead.

---

## Part II: Precision-Adaptive Optimizer States

### 3. Per-Parameter Optimizer Precision Selection

bitsandbytes uses a fixed threshold: parameters with <4096 elements stay at 32-bit, everything else goes to 8-bit. This is a crude heuristic. NSL can do much better.

#### 3.1 Compile-Time Sensitivity Analysis

When weights are available, the compiler computes a **quantization sensitivity score** per parameter:

```
sensitivity(θ) =
    spectral_condition(W)     # condition number of weight matrix — high =
sensitive
  × gradient_magnitude_est(θ) # estimated gradient scale from weight analysis
  × position_criticality(l)   # early/late layers more sensitive
  / element_count(θ)          # larger parameters tolerate more noise (averaging
effect)
```

Parameters are then assigned precision:

| Sensitivity | Precision | Memory per param |
|---|---|---|
| High (norms, embeddings, first/last layer) | FP32 (m and v) | 8 bytes |
| Medium (attention projections) | FP16 m, FP32 v | 6 bytes |
| Low (FFN gate/up/down in middle layers) | INT8 m, FP16 v | 3 bytes |
| Very low (provably stable via WCET) | INT8 m, INT8 v | 2 bytes |

**Memory comparison (NSLCoder-50M, 48.8M params):**

| Strategy | Optimizer State Memory |
|---|---|
| FP32 Adam (standard) | 390 MB (8 bytes/param) |
| INT8 Adam (bitsandbytes uniform) | 97.5 MB (2 bytes/param) |
| **CPDT precision-adaptive** | **~65 MB** (1.3 bytes/param average) |

The additional 30% savings over uniform INT8 comes from using INT8 for BOTH m and v on low-sensitivity parameters (FFN middle layers), which bitsandbytes doesn't support because it can't assess sensitivity.

### 3.2 FASE Integration: Fused Quantized Optimizer

From our FASE research (Paper 3), the optimizer step is fused into the backward pass. CPDT extends this with per-layer quantization precision:

```
Compiled backward + quantized optimizer (layer 5, low sensitivity):

  g_5 = compute_gradient(layer_5)           # FP16 gradient

  # Fused: dequant INT8 → FP32, optimizer step, quant FP32 → INT8
  m_5_fp32 = dequant_int8(m_5_int8)         # 3 cycles (LUT + denorm)
  v_5_fp16 = dequant_fp16(v_5_fp16_stored)  # 1 cycle (reinterpret)

  m_5_fp32 = β₁ × m_5_fp32 + (1-β₁) × g_5.to_fp32()
  v_5_fp16 = β₂ × v_5_fp16 + (1-β₂) × (g_5² ).to_fp16()

  θ_5 -= lr × (m̂_5 / (√v̂_5.to_fp32() + ε) + wd × θ_5)

  m_5_int8 = quant_int8_blockwise(m_5_fp32)  # blockwise dynamic quantization
  v_5_fp16_stored = v_5_fp16

  FREE: g_5                                  # gradient never stored
```

This entire sequence is generated as a single fused kernel per layer. The quant/dequant operations happen in registers — no additional HBM traffic.

### 3.3 Stochastic Rounding at Compile Time

Q-Adam-mini discovered that embedding layers need stochastic rounding for INT8 stability. In bitsandbytes, this is a runtime flag. In CPDT, the compiler knows which layers are embeddings (from the `model` AST) and automatically generates stochastic rounding kernels for those layers and deterministic rounding for everything else.

---

# Part III: Static Expert Routing for MoE

## 4. Compile-Time MoE Optimization

NSL has MoE support (M32) with the `@moe` decorator. CPDT adds three compiler-native optimizations.

### 4.1 Expert Capacity from Cost Model

The optimal capacity factor (how many tokens each expert can process) depends on hardware:

- **Compute-bound regime** (large batch, short seq): Low capacity factor is fine — experts process fewer tokens but each with full compute
- **Memory-bound regime** (small batch, long seq): Higher capacity factor better — fill the idle ALUs

The roofline model determines this automatically:

```
capacity_factor = max(1.0, roofline_slack(expert_ffn) × top_k / n_experts)
```

### 4.2 Dead Expert Pruning (with --weights)

When pre-trained MoE weights are available, the compiler analyzes the router's weight matrix to identify dead or near-dead experts:

```
router_weights: [d_model, n_experts]   # gating network

For each expert e:
  router_affinity(e) = ‖router_weights[:, e]‖₂
  # Low affinity → this expert is rarely selected
```

Experts with affinity below a threshold are pruned via CEP (Paper 5). The compiled binary contains fewer expert kernels, and the router's output dimension is reduced. This is true structural MoE pruning — dead experts don't just receive zero tokens, they don't exist in the binary.

### 4.3 Expert Placement Planning

For multi-GPU MoE training, the compiler plans expert placement:

```
Given: 8 experts, 4 GPUs, expert sizes, compute costs

Placement options:
  A) 2 experts per GPU (balanced)
  B) Expert-parallel: all tokens routed to expert's home GPU
  C) Hybrid: popular experts replicated, rare experts sharded

CPDT evaluation:
  For each placement:
    1. Communication cost: all-to-all dispatch volume
    2. Load balance: expected utilization per GPU
    3. Memory per GPU: expert parameters + activations
    4. Compute time: per-expert forward/backward (from cost model)

  Select: minimum total step time subject to memory constraints
```

The placement is a compile-time constant. The all-to-all communication pattern is generated as part of the compiled forward/backward function, not as a runtime dispatch.

---

## 5. The Joint Optimization

CPDT's key insight: ZeRO sharding, optimizer precision, and expert placement are **interdependent** decisions that should be optimized jointly.

### 5.1 Why Joint Matters

- **ZeRO stage affects optimizer precision:** Higher ZeRO stages shard optimizer states, making 8-bit quantization less impactful (each shard is already smaller). CPDT factors this in when selecting precision.
- **MoE expert placement affects ZeRO mesh:** Experts already create a sharding pattern. ZeRO's mesh should align with expert placement to avoid conflicting communication patterns.
- **Optimizer precision affects memory budget:** Cheaper optimizer states free memory for larger activation budgets, which in turn affects whether full ZeRO-3 is needed or ZeRO-1 suffices.

### 5.2 The Joint Optimization Problem

```
minimize: total_step_time(zero_config, precision_config, expert_placement)
subject to:
  memory_per_gpu(zero_config, precision_config, expert_placement) ≤ GPU_memory
  numerical_stability(precision_config) ≥ threshold
  expert_load_balance(expert_placement) ≥ min_utilization
```

The compiler solves this via iterative search:

1. Fix expert placement → optimize ZeRO + precision (fast: ~200ms)
2. Fix ZeRO + precision → optimize expert placement (fast: ~100ms)
3. Iterate until convergence (typically 3-5 iterations: ~1 second)

---

## 6. NSL Language Surface

### 6.1 Automatic Planning

nsl

```
# CPDT plans everything automatically from the model + cluster config
@cpdt(
    cluster = { gpus = 8, intra_bw = "900GB/s", inter_bw = "100GB/s" },
    target_memory = "80%",   # use up to 80% of GPU memory
    precision = auto         # compiler selects per-parameter precision
)
train(model=m, epochs=1, grad_clip=1.0):
    optimizer: AdamW(lr=0.0003, weight_decay=0.1)
    step(batch):
        let logits = m.forward_train(batch.input_ids, true)
        let loss = cross_entropy(logits, batch.labels)
```

### 6.2 CLI

bash

```
$ nsl build --cpdt-plan --cluster 8xH100-NVLink --weights model.safetensors model.ns

=== CPDT Training Plan ===
Cluster: 8× H100-SXM (80GB each, NVLink 900GB/s intra, IB 100GB/s inter)
Model: NSLCoder-50M (48.8M params)

ZeRO Configuration:
  Parameters: replicated (s_p=1) — model fits in single GPU
  Gradients: reduce-scatter intra-node (s_g=8)
  Optimizer states: sharded across all GPUs (s_os=8)
  Effective: ZeRO-1 equivalent with intra-node gradient reduction
  Communication schedule: 127 async reducescatter ops overlapped with backward

Optimizer Precision:
  embed (49K vocab × 512): FP32 m+v, stochastic rounding (2.0 MB)
  blocks.0-1 (attention+FFN): FP16 m, FP32 v (1.8 MB each)
  blocks.2-5 (middle layers): INT8 m, FP16 v (0.9 MB each)
  blocks.6-7 (late layers): FP16 m, FP32 v (1.8 MB each)
  norm (512 params): FP32 m+v (4 KB)
  Total optimizer memory: 14.6 MB / GPU (vs 48.8 MB uniform FP32)

Memory per GPU: 1.2 GB (model) + 14.6 MB (optimizer/8) + 0.8 GB (activations) = 2.0
Estimated step time: 4.2ms (vs 5.8ms naive DDP)
MFU: 67% (vs 48% naive DDP)
```

---

## 7. Composition with Previous Techniques

| Technique | Interaction with CPDT |
|---|---|
| **WRGA** (PEFT) | CPDT plans sharding for only the trainable adapter parameters; frozen params skip communication |
| **CSHA** (Attention) | CSHA kernels fused with CPDT's allgather-compute overlap schedule |
| **FASE** (Gradient Accum) | FASE's fused optimizer step uses CPDT's per-layer precision config |
| **PCA** (Packing) | PCA's segment-ID kernels distributed across CPDT's data-parallel mesh |
| **CEP** (Pruning/NAS) | CEP evaluates candidates using CPDT's planned memory/comm costs |

---

## 8. Why PyTorch Cannot Do This

1. **No joint optimization.** DeepSpeed, FSDP, and bitsandbytes are configured independently. There is no tool that jointly optimizes ZeRO stage + optimizer precision + expert placement.
2. **No compile-time communication scheduling.** PyTorch's communication hooks (`register_comm_hook`, `register_post_accumulate_grad_hook`) fire at runtime in Python. NSL generates communication calls as part of the compiled backward IR.
3. **No per-parameter adaptive precision.** bitsandbytes offers per-parameter precision overrides, but the user must manually specify them. CPDT selects precision automatically using spectral analysis.
4. **No analytical ZeRO evaluation.** DeepSpeed's auto-tuning requires runtime profiling. AMSP requires a communication profiler. CPDT evaluates configurations analytically from the cost model.
5. **No expert-aware sharding.** DeepSpeed's MoE module and ZeRO operate independently. CPDT aligns expert placement with ZeRO mesh topology.

---

## 9. Implementation Plan

| Component | Location | LOC (est.) | Description |
|---|---|---|---|
| ZeRO planner | `nsl-codegen/src/cpdt_zero.rs` | 600 | Analytical ZeRO config search |
| Comm scheduler | `nsl-codegen/src/cpdt_comm.rs` | 500 | Per-layer allgather/reducescatter scheduling |
| Precision selector | `nsl-codegen/src/cpdt_precision.rs` | 400 | Per-parameter optimizer precision from spectral analysis |
| Quantized optimizer codegen | `nsl-codegen/src/cpdt_optim.rs` | 500 | Fused 8/16/32-bit optimizer step generation |
| Expert planner | `nsl-codegen/src/cpdt_expert.rs` | 400 | MoE expert placement optimization |
| Joint solver | `nsl-codegen/src/cpdt_joint.rs` | 300 | Iterative joint optimization |
| Runtime comm | `nsl-runtime/src/cpdt_comm.rs` | 400 | NCCL/SharedMem wrappers for planned schedule |

**Dependencies:** cost_model.rs, memory_planner.rs, weight_aware.rs, pipeline.rs, tensor_parallel.rs, moe.rs, zero.rs

---

## 10. Conclusion

CPDT completes the six-paper compiler-native training optimization suite:

| Paper | Domain | Core Insight |
|---|---|---|
| **WRGA** | PEFT | Adapter placement is a compilation decision |
| **CSHA** | Attention | Fusion boundaries are compiler-removable |
| **FASE** | Gradient Accumulation | Accumulation + optimizer can be fused at compile time |
| **PCA** | Sequence Packing | Document masks baked into kernel control flow |
| **CEP** | Pruning + NAS | The compiler IS the evaluation oracle |
| **CPDT** | **Distributed Training** | **Sharding, precision, and placement are a joint compile-time optimization** |

The unifying thesis across all six: **training optimization is a compilation problem.** Every decision that PyTorch/DeepSpeed/FSDP makes at runtime — where to place adapters, how to fuse attention, when to accumulate gradients, how to pack sequences, what to prune, how to shard, what precision to use — NSL makes at compile time. The compiled binary IS the training plan.

---

## References


1. Rajbhandari, S., et al. "ZeRO: Memory Optimizations Toward Training Trillion Parameter Models." SC 2020.
2. Zhao, Y., et al. "PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel." VLDB 2023.
3. Dettmers, T., et al. "8-bit Optimizers via Block-wise Quantization." ICLR 2022.
4. Tianwei, Z., et al. "AMSP: Reducing Communication Overhead of ZeRO for Efficient LLM Training." 2024.
5. Fedus, W., et al. "Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity." JMLR 2022.
6. DeepSpeed Team. "DeepCompile: Compiler-Level Optimizations for Distributed Training." 2025.